

TRÌNH BIÊN DỊCH

Chương 2. Phân tích cú pháp-LL(1) tt

Phạm Nguyễn Khang
BM. Khoa học máy tính

pnkhang@cit.ctu.edu.vn

CẦN THƠ, 12 - 2012

```
#include ...  
int main() {  
    ...  
}
```



Phân tích từ vựng

Phân tích cú pháp

Phân tích ngữ nghĩa

Sinh mã trung gian

Tối ưu mã trung gian

Sinh mã

Tối ưu



```
0001001000  
1011011100  
1100101111  
1000011111
```

Giải thuật tính tập FIRST

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**,
 - Gán **FIRST(A) = { t | A → tω }**
- Lặp lại cho đến khi bảng T không thay đổi nữa:
 - Với mỗi ký hiệu chưa kết thúc **A**,
 - Với mỗi luật sinh có dạng **A → Bω**,
 - Gán **FIRST(A) = FIRST(A) ∪ FIRST(B)**

Giải thuật tính FIRST trong trường hợp luật sinh rỗng

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**
 - Gán **FIRST(A) = { t | A → tω }**
- Với mỗi luật sinh **A → ε**
 - Thêm **ε** vào **FIRST(A)**
- Lặp lại cho đến khi không thay đổi nữa:
 - Với mỗi luật sinh **A → α**,
 - Gán **FIRST(A) = FIRST(A) ∪ FIRST*(α)**

FOLLOW

- Với luật sinh rỗng, ta phải “nhìn xuyên qua” ký hiệu chưa kết thúc để xem sau nó là gì.
- Tập **FOLLOW** biểu diễn tập các ký hiệu kết thúc có thể đi sau một ký hiệu không kết thúc nào đó.
- Chặt chẽ hơn, ta định nghĩa:
 - **FOLLOW(A) = { t | S $\Rightarrow^*$ aAtw }** trong đó **S** là kí hiệu bắt đầu của văn phạm.

Tính FOLLOW

- Khởi tạo, với mỗi ký hiệu chưa kết thúc **A**
 - Gán $\text{FOLLOW}(\mathbf{A}) = \{ \mathbf{t} \mid \mathbf{B} \rightarrow \mathbf{aAt\omega} \}$
- Thêm **\$** vào $\text{FOLLOW}(\mathbf{S})$, **S** là ký hiệu vắt đầu.
- Lặp lại cho đến khi không còn thay đổi:
 - Với mỗi luật sinh dạng $\mathbf{B} \rightarrow \mathbf{aA\omega}$,
 - Gán $\text{FOLLOW}(\mathbf{A}) = \text{FOLLOW}(\mathbf{A}) \cup \text{FIRST}^*(\omega) - \{ \epsilon \}$.
 - Với mỗi luật sinh dạng $\mathbf{B} \rightarrow \mathbf{aA\omega}$ trong đó $\epsilon \in \text{FIRST}^*(\omega)$,
 - Gán $\text{FOLLOW}(\mathbf{A}) = \text{FOLLOW}(\mathbf{A}) \cup \text{FOLLOW}(\mathbf{B})$.

Giải thuật xây dựng bảng LL(1)

- Tính **FIRST(A)** và **FOLLOW(A)** cho tất cả các ký hiệu chưa kết thúc **A**.
- Với mỗi luật sinh **A** $\rightarrow$ **ω** ,
 - Với mỗi ký hiệu kết thúc **t** $\in$ **FIRST*(ω)**,
 - Gán **T[A, t] = ω** .
 - Chú ý: **ϵ** không phải là ký hiệu kết thúc.
- Với mỗi luật sinh **A** $\rightarrow$ **ω** ,
 - Nếu **$\epsilon \in$ FIRST*(ω)**,
 - Gán **T[A, t] = ω** cho tất cả các **t** $\in$ **FOLLOW(A)**.

Giải thuật xây dựng bảng LL(1)

- Lạm dụng ký hiệu một chút ta có thể viết lại giải thuật gọn hơn như sau:
 - Tính **FIRST(A)** và **FOLLOW(A)** cho tất cả các ký hiệu chưa kết thúc **A**.
 - Với mỗi luật sinh **A** $\rightarrow \omega$
 - Với mỗi ký hiệu kết thúc $t \in \mathbf{FIRST}^*(\omega\mathbf{FOLLOW(A)})$,
 - Gán $T[A, t] = \omega$.

Hạn chế của giải thuật LL(1)

- Xét ví dụ sau

$$\mathbf{A} \rightarrow \mathbf{A}b \mid c$$

- $\text{FIRST}(\mathbf{A}) = \{c\}$
- Tuy nhiên ta không thể xây dựng bảng LL(1) cho văn phạm trên.
- Tại sao ?

Hạn chế của giải thuật LL(1)

- Xét ví dụ sau

$$A \rightarrow Ab \mid c$$

- $\text{FIRST}(A) = \{c\}$
- Tuy nhiên ta không thể xây dựng bảng LL(1) cho văn phạm trên.
- Tại sao ?

	b	c
A		$A \rightarrow Ab$ $A \rightarrow c$

Hạn chế của giải thuật LL(1)

- Xét ví dụ sau

$$A \rightarrow Ab \mid c$$

- $\text{FIRST}(A) = \{c\}$
- Tuy nhiên ta không thể xây dựng bảng LL(1) cho văn phạm trên.

- Tại sao ?

	b	c
A		$A \rightarrow Ab$ $A \rightarrow c$

- Không thể dự đoán được luật sinh nào sẽ được sử dụng
- **Đụng độ FIRST/FIRST**

Khử đệ quy trái

- Tổng quát, đệ quy trái có thể chuyển thành đệ quy phải bằng các quy tắc sau đây.
- Xét văn phạm
 - $A \rightarrow A\omega \mid \alpha$
 - Sẽ sinh ra α , theo sau là 0 hoặc nhiều ω .
- Ta có thể viết lại như sau:
 - $A \rightarrow \alpha B$
 - $B \rightarrow \epsilon \mid \omega B$

Một ví dụ khác

- Xét văn phạm sau:
- $\mathbf{E} \rightarrow \mathbf{T}$
- $\mathbf{E} \rightarrow \mathbf{T} + \mathbf{E}$
- $\mathbf{T} \rightarrow \mathbf{int}$
- $\mathbf{T} \rightarrow (\mathbf{E})$
- $\text{FIRST}(\mathbf{E}) = \{ \mathbf{int}, (\}$
- $\text{FIRST}(\mathbf{T}) = \{ \mathbf{int}, (\}$
- Tại sao văn phạm này không phải là văn phạm LL(1) ?

Một ví dụ khác

- Xét văn phạm sau:

- **$E \rightarrow T$**
- **$E \rightarrow T + E$**

Chọn luật sinh
nào ?

- **$T \rightarrow \text{int}$**
- **$T \rightarrow (E)$**
- **$\text{FIRST}(E) = \{ \text{int}, (\}$**
- **$\text{FIRST}(T) = \{ \text{int}, (\}$**
- Tại sao văn phạm này không phải là văn phạm LL(1) ?

Tạo nhân tố trái

E $\rightarrow$ **T**

E $\rightarrow$ **T** + **E**

T $\rightarrow$ int

T $\rightarrow$ (**E**)

Tạo nhân tố trái

$E \rightarrow T\epsilon$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

Tạo nhân tố trái

E $\rightarrow$ **TY**

T $\rightarrow$ **int**

T $\rightarrow$ (**E**)

Tạo nhân tố trái

E $\rightarrow$ **TY**

T $\rightarrow$ **int**

T $\rightarrow$ (**E**)

Y $\rightarrow$ **+** **E**

Y $\rightarrow$ ϵ

Tạo nhân tố trái

E $\rightarrow$ **TY** **1**

T $\rightarrow$ **int** **2**

T $\rightarrow$ (**E**) **3**

Y $\rightarrow$ **+** **E** **4**

Y $\rightarrow$ ϵ **5**

Tạo nhân tố trái

E	$\rightarrow$	T Y	1
T	$\rightarrow$	int	2
T	$\rightarrow$	(E)	3
Y	$\rightarrow$	+ E	4
Y	$\rightarrow$	ϵ	5

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
FOLLOW		
E	T	Y

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
	int (	
FOLLOW		
E	T	Y

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
	int (	+ ε
FOLLOW		
E	T	Y

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y

Tạo nhân tố trái

E $\rightarrow$ **TY** **1**
T $\rightarrow$ **int** **2**
T $\rightarrow$ **(E)** **3**
Y $\rightarrow$ **+ E** **4**
Y $\rightarrow$ **ϵ** **5**

FIRST		
E	T	Y
int (	int (	+ ϵ
FOLLOW		
E	T	Y
\$		

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y
\$)		

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y
\$)	+	

Tạo nhân tố trái

E $\rightarrow$ **T****Y** **1**
T $\rightarrow$ **int** **2**
T $\rightarrow$ **(E)** **3**
Y $\rightarrow$ **+** **E** **4**
Y $\rightarrow$ **ϵ** **5**

FIRST		
E	T	Y
int (	int (	+ ϵ
FOLLOW		
E	T	Y
\$)	+	\$)

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

	int	(	)	+	\$
E					
T					
Y					

Tạo nhân tố trái

$E \rightarrow TY$ **1**
 $T \rightarrow \text{int}$ **2**
 $T \rightarrow (E)$ **3**
 $Y \rightarrow + E$ **4**
 $Y \rightarrow \epsilon$ **5**

FIRST		
E	T	Y
int (	int (	+ ϵ
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

	int	(	)	+	\$
E	1	1			
T					
Y					

Tạo nhân tố trái

E → TY	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int (	int (	+ ε
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

	int	(	)	+	\$
E	1	1			
T	2	3			
Y					

Tạo nhân tố trái

$E \rightarrow TY$ **1**
 $T \rightarrow \text{int}$ **2**
 $T \rightarrow (E)$ **3**
 $Y \rightarrow + E$ **4**
 $Y \rightarrow \epsilon$ **5**

FIRST		
E	T	Y
int (	int (	+ ϵ
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

	int	(	)	+	\$
E	1	1			
T	2	3			
Y				4	

Tạo nhân tố trái

$E \rightarrow TY$ **1**
 $T \rightarrow \text{int}$ **2**
 $T \rightarrow (E)$ **3**
 $Y \rightarrow + E$ **4**
 $Y \rightarrow \epsilon$ **5**

FIRST		
E	T	Y
int (	int (	+ ϵ
FOLLOW		
E	T	Y
\$)	+ \$)	\$)

	int	(	)	+	\$
E	1	1			
T	2	3			
Y			5	4	5

Văn phạm LL(1)

- Văn phạm G được gọi là LL(1) khi và chỉ khi với mọi cặp luật sinh có dạng **$A \rightarrow \omega_1$ and $A \rightarrow \omega_2$**
 - 2 tập:
 - $\text{FIRST}(\omega_1 \text{ FOLLOW}(A))$
 - $\text{FIRST}(\omega_2 \text{ FOLLOW}(A))$không giao nhau.
- Điều này tương đương với việc nói rằng không có đụng độ xảy ra trong bảng phân tích.

Ưu điểm của LL(1)

- Có thể cài đặt nhanh chóng bằng bảng và stack
- Có thể cài đặt bằng phương pháp **đệ quy lùi (recursive descent)**:
 - Định nghĩa hàm cho mỗi ký hiệu chưa kết thúc.
 - Các hàm sẽ gọi nhau dựa trên token nhìn trước.
- Xem demo

Ưu điểm của LL(1)

- Cả hai phương pháp cài đặt đều nhanh
- Độ phức tạp thời gian là $O(n |G|)$, trong đó n là chiều dài chuỗi nhập và $|G|$ là kích thước của văn phạm.

Tổng kết

- Phân tích từ trên xuống cố gắng dẫn xuất ra chương trình đầu vào từ ký hiệu bắt đầu..
- BFS trái nhất là một cách tiếp cận từ trên xuống; chỉ có ý nghĩa lý thuyết, trong thực tế khi sử dụng.
- DFS trái nhất cũng là một tiếp cận từ trên xuống và cũng ít được sử dụng trong thực tế.
- LL(1) phân tích bằng cách đọc từ trái sang phải, nhìn trước một token để tìm dẫn xuất trái nhất.

Tổng kết

- FIRST chứa các ký hiệu kết thúc mà có thể nằm ở đầu chuỗi được sinh ra từ một ký hiệu chưa kết thúc.
- FOLLOW chứa các ký hiệu kết thúc mà có thể theo sau một ký hiệu chưa kết thúc.
- Độ quy trái và khả năng tạo nhân tố trái là nguyên nhân làm cho LL(1) không hoạt động được. Có thể khử độ quy trái và tạo nhân tố trái trong một số trường hợp.